

React Props Explanation



You said:

ChatGPT said:

React Props

React में **Props** का मतलब होता है **Properties**।

Props का use एक component से दूसरे component में **data भेजने** के लिए किया जाता है.

Simple words में:

👉 **Props = Component के arguments**

जैसे JavaScript function में parameters pass करते हैं, वैसे ही React component में props pass करते हैं।

Why Props?

Props use होते हैं:

- Data pass करने के लिए
- Dynamic UI बनाने के लिए

- Reusable components बनाने के लिए

Basic Example

Parent Component

```
JavaScriptfunction App() {  
  return (  
    <div>  
      <Student name="Aamir" age="21" />  
      <Student name="Rahul" age="22" />  
    </div>  
  );  
}
```

Child Component

```
JavaScriptfunction Student(props) {  
  return (  
    <div>  
      <h1>Name: {props.name}</h1>  
      <h2>Age: {props.age}</h2>  
    </div>  
  );  
}
```

Output

Name: Aamir
Age: 21

Name: Rahul
Age: 22

How Props Work

```
JavaScript<Student name="Aamir" age="21" />
```

यहाँ:

- **name="Aamir"** → prop

- `age="21" → prop`

React automatically object बनाता है:

```
JavaScriptprops = {  
  name: "Aamir",  
  age: "21"  
}
```

Props Using Destructuring

Better way:

```
JavaScriptfunction Student({ name, age }) {  
  return (  
    <div>  
      <h1>{name}</h1>  
      <h2>{age}</h2>  
    </div>  
  );  
}
```

Passing Array in Props

```
JavaScriptfunction App() {  
  const fruits = ["Apple", "Mango", "Banana"];  
  
  return <FruitList items={fruits} />;  
}
```

```
JavaScriptfunction FruitList({ items }) {  
  return (  
    <ul>  
      {items.map((fruit) => (  
        <li>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

Passing Object in Props

```
JavaScriptfunction App() {  
  const user = {  
    name: "Aamir",  
    city: "Delhi"  
  };  
  
  return <Profile user={user} />;  
}
```

```
JavaScriptfunction Profile({ user }) {  
  return (  
    <div>  
      <h1>{user.name}</h1>  
      <h2>{user.city}</h2>  
    </div>  
  );  
}
```

Passing Function as Props

```
JavaScriptfunction App() {  
  function greet() {  
    alert("Hello");  
  }  
  
  return <Button click={greet} />;  
}
```

```
JavaScriptfunction Button({ click }) {  
  return <button onClick={click}>Click</button>;  
}
```

Children Props

React में component के बीच का content **children** कहलाता है।


```
JavaScriptfunction App() {
  return (
    <Card>
      <h1>Hello</h1>
    </Card>
  );
}
```

```
JavaScriptfunction Card(props) {
  return (
    <div>
      {props.children}
    </div>
  );
}
```

Important Rules

1. Props are Read Only

✗ Wrong

```
JavaScriptprops.name = "Ali";
```

✓ Correct

Props को change नहीं करते।

2. Parent → Child Data Flow

Data हमेशा ऊपर से नीचे जाता है।

Parent → Child

Difference Between Props and State

Props	State
Read only	Mutable
Parent sends data	Component manages data

Props**State**

Used for communication Used for dynamic changes

Real Life Analogy

Imagine:

Mobile = Component
Charger = Props

Mobile को charger बाहर से मिलता है।

वैसे ही component को data बाहर से मिलता है → Props।

Interview Questions

1. What are props in React?

Props are used to pass data from parent component to child component.

2. Are props mutable?

No, props are immutable (read-only).

3. Can we pass functions in props?

Yes.

4. What is props drilling?

Passing props through multiple components unnecessarily.

Short Notes

Props:

- Properties
- Used to pass data
- Parent to child communication
- Read only
- Makes component reusable

ChatGPT said:

React Activity on Props, Arrays, Conditional Rendering & Dynamic Styling

Activity 1: Student Card using Props

Task

Create a **Student** component and pass:

- name
- age
- city

using props.

Expected Output

```
Name: Aamir  
Age: 21  
City: Delhi
```

Solution

```
JavaScriptfunction Student({ name, age, city }) {  
  return (  
    <div>  
      <h1>Name: {name}</h1>  
      <h2>Age: {age}</h2>  
      <h3>City: {city}</h3>  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <Student  
      name="Aamir"  
      age={21}  
      city="Delhi"  
    />  
  );  
}
```

Activity 2: Render Array Dynamically

Task

Render fruits using `map()`.

Expected Output

```
Apple  
Mango  
Banana
```

Solution

```
JavaScriptfunction App() {  
  
  const fruits = [  
    "Apple",  
    "Mango",  
    "Banana"  
  ];  
  
  return (  
    <div>  
      {fruits.map((fruit, index) => (  
        <h1 key={index}>{fruit}</h1>  
      ))}  
    </div>  
  );  
}
```

Activity 3: Product Cards

Task

Create dynamic product cards using arrays.

Solution

```
JavaScriptfunction Product({ name, price }) {  
  return (  
    <div>  
      <h1>{name}</h1>  
      <h2>{price}</h2>  
    </div>  
  );  
}  
  
function App() {  
  
  const products = [  
    {  
      id: 1,  
      name: "iPhone",  
      price: 90000  
    },  
    {  
      id: 2,  
      name: "Laptop",  
      price: 50000  
    }  
  ];  
  
  return (  
    <div>  
      {products.map((product) => (  
        <Product  
          key={product.id}  
          name={product.name}  
          price={product.price}  
        />  
      ))}  
    </div>  
  );  
}
```

Activity 4: Conditional Rendering

Task

Show:

- "Welcome" if logged in

- "Please Login" if not

Solution

```
JavaScriptfunction App() {  
  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h1>Welcome</h1>  
      ) : (  
        <h1>Please Login</h1>  
      )}  
    </div>  
  );  
}
```

Activity 5: Dynamic Button Styling

Task

Button color should change dynamically.

Solution

```
JavaScriptfunction App() {  
  
  const active = true;  
  
  return (  
    <button  
      style={{  
        backgroundColor: active  
          ? "blue"  
          : "gray",  
        color: "white",  
        padding: "10px"  
      }}  
    >  
      Click Me  
    </button>  
  );  
}
```

Activity 6: Even Numbers Rendering

Task

Render only even numbers from array.

Solution

```
JavaScriptfunction App() {  
  
  const numbers = [1, 2, 3, 4, 5, 6];  
  
  return (  
    <div>  
      {numbers  
        .filter((num) => num % 2 === 0)  
        .map((num) => (  
          <h1 key={num}>{num}</h1>  
        ))}  
    </div>  
  );  
}
```

Output

```
2  
4  
6
```

Activity 7: Dynamic User List

Task

Create user cards dynamically.

Solution

```
JavaScriptfunction User({ name, city }) {  
  return (  
    <div>  
      <h1>{name}</h1>  
      <h2>{city}</h2>  
    </div>  
  );  
}  
  
function App() {  
  
  const users = [  
    {  
      id: 1,  
      name: "Aamir",  
      city: "Delhi"  
    },  
    {  
      id: 2,  
      name: "Rahul",  
      city: "Mumbai"  
    }  
  ];  
  
  return (  
    <div>  
      {users.map((user) => (  
        <User  
          key={user.id}  
          name={user.name}  
          city={user.city}  
        />  
      ))}  
    </div>  
  );  
}
```

Activity 8: Toggle Theme

Task

Create dark/light theme toggle.

Solution

```
JavaScriptimport { useState } from "react";

function App() {

  const [dark, setDark] = useState(false);

  return (
    <div
      style={{
        backgroundColor: dark
          ? "black"
          : "white",
        color: dark
          ? "white"
          : "black",
        height: "100vh"
      }}
    >
      <button
        onClick={() => setDark(!dark)}
      >
        Toggle Theme
      </button>

      <h1>Hello React</h1>
    </div>
  );
}
```

Practice Tasks for You

Easy

1. Render 5 cities dynamically
2. Create user profile using props
3. Show "Admin" using conditional rendering

Medium

4. Create product cards using arrays
5. Make dynamic navbar

6. Render only odd numbers

Advanced

7. Todo List

8. Dynamic theme switcher

9. Shopping cart UI

10. Dynamic dashboard cards

Mini Challenge

Create:

Netflix style movie cards

using:

- props
- arrays
- map()
- dynamic styling

Final Revision Notes

Topics Covered:

- Props
- Props Drilling
- Passing Arrays
- Rendering Arrays
- Conditional Rendering
- Dynamic Components
- Dynamic Styling

You said:

install react developer tools

ChatGPT said:

Install React Developer Tools

What is React Developer Tools?

React Developer Tools is a browser extension used to:

- inspect React components
- view props and state
- debug React apps
- analyze component tree

It works in:

- Google Chrome
- Microsoft Edge
- Firefox

Install in Google Chrome

Step 1

Open:

[Chrome Web Store – React Developer Tools](#)

Step 2

Click:

Add to Chrome

Step 3

Click:

Add Extension

Install in Microsoft Edge

Open:

[Microsoft Edge Add-ons – React Developer Tools](#)

Then click:

Get

Install in Firefox

Open:

[Firefox Add-ons – React Developer Tools](#)

Then click:

Add to Firefox

How to Open React DevTools

Step 1

Run your React app.

Example:

Bashnpm start

OR

Bashnpm run dev

Step 2

Open browser.

Step 3

Right click → Inspect

OR press:

F12

Step 4

You will see new tabs:

⌘ Components
⌘ Profiler

Components Tab

Used to:

- inspect components
- view props
- view state
- check hooks

Profiler Tab

Used to:

- analyze performance
- detect slow rendering
- optimize app

Example

Suppose component:

```
JavaScriptfunction User({ name }) {  
  return <h1>{name}</h1>;  
}
```

React DevTools can show:

```
User  
props:  
name: "Aamir"
```

Why React Developer Tools Important?

Very useful for:

- debugging
- interview preparation
- learning React
- professional development

Common Problem

DevTools Not Showing?

Solution

1. Restart browser
2. Reload React app
3. Make sure React app is running
4. Disable browser cache
5. Reinstall extension

Interview Question

What is React Developer Tools?

A browser extension used to inspect and debug React applications.

Final Short Notes

React Developer Tools:

- Browser extension
- Used for debugging React apps
- Shows:
 - components
 - props
 - state
 - hooks
- Tabs:
 - Components
 - Profiler

You said:

ChatGPT said:

React Activity: Amazon Product Cards

Goal

Create Amazon-style product cards using:

- Props
- Arrays
- `map()`
- Dynamic Components

- Dynamic Styling

Final Output Idea

```
-----  
| Product Image   |  
| Product Name   |  
| Price          |  
| Rating ★       |  
| Add to Cart Button |  
-----
```

Step 1: Create Product Data

```
JavaScriptconst products = [  
  {  
    id: 1,  
    name: "iPhone 15",  
    price: 90000,  
    rating: 4.5,  
    image:  
      "https://via.placeholder.com/150"  
  },  
  
  {  
    id: 2,  
    name: "Laptop",  
    price: 50000,  
    rating: 4.2,  
    image:  
      "https://via.placeholder.com/150"  
  },  
  
  {  
    id: 3,  
    name: "Headphones",  
    price: 3000,  
    rating: 4.8,  
    image:  
      "https://via.placeholder.com/150"  
  }  
];
```

Step 2: Create Product Card Component

```
JavaScriptfunction ProductCard({
  name,
  price,
  rating,
  image
}) {

  return (
    <div style={styles.card}>

      <img
        src={image}
        alt={name}
        style={styles.image}
      />

      <h2>{name}</h2>

      <h3>₹ {price}</h3>

      <p>★ {rating}</p>

      <button style={styles.button}>
        Add to Cart
      </button>

    </div>
  );
}
```

Step 3: Render Cards Dynamically

```
JavaScriptfunction App() {  
  
  const products = [  
    {  
      id: 1,  
      name: "iPhone 15",  
      price: 90000,  
      rating: 4.5,  
      image:  
        "https://via.placeholder.com/150"  
    },  
  
    {  
      id: 2,  
      name: "Laptop",  
      price: 50000,  
      rating: 4.2,  
      image:  
        "https://via.placeholder.com/150"  
    },  
  
    {  
      id: 3,  
      name: "Headphones",  
      price: 3000,  
      rating: 4.8,  
      image:  
        "https://via.placeholder.com/150"  
    }  
  ];  
  
  return (  
  
    <div style={styles.container}>  
  
      {products.map((product) => (  
  
        <ProductCard  
          key={product.id}  
          name={product.name}  
          price={product.price}  
          rating={product.rating}  
          image={product.image}  
        />  
  
      ))}  
  
    </div>
```

```
);  
}
```

Step 4: Add Styling

```
JavaScriptconst styles = {  
  
  container: {  
    display: "flex",  
    gap: "20px",  
    flexWrap: "wrap",  
    justifyContent: "center"  
  },  
  
  card: {  
    width: "220px",  
    border: "1px solid #ddd",  
    borderRadius: "10px",  
    padding: "15px",  
    textAlign: "center",  
    boxShadow: "0 2px 8px rgba(0,0,0,0.2)"  
  },  
  
  image: {  
    width: "100%",  
    borderRadius: "10px"  
  },  
  
  button: {  
    backgroundColor: "orange",  
    border: "none",  
    padding: "10px",  
    width: "100%",  
    cursor: "pointer",  
    borderRadius: "5px",  
    marginTop: "10px"  
  }  
};
```

Complete Final Code

```
JavaScriptfunction ProductCard({
  name,
  price,
  rating,
  image
}) {

  return (
    <div style={styles.card}>

      <img
        src={image}
        alt={name}
        style={styles.image}
      />

      <h2>{name}</h2>

      <h3>₹ {price}</h3>

      <p>★ {rating}</p>

      <button style={styles.button}>
        Add to Cart
      </button>

    </div>
  );
}

function App() {

  const products = [
    {
      id: 1,
      name: "iPhone 15",
      price: 90000,
      rating: 4.5,
      image:
        "https://via.placeholder.com/150"
    },

    {
      id: 2,
      name: "Laptop",
      price: 50000,
      rating: 4.2,
      image:
```

```
      "https://via.placeholder.com/150"
    },

    {
      id: 3,
      name: "Headphones",
      price: 3000,
      rating: 4.8,
      image:
        "https://via.placeholder.com/150"
    }
  ];

  return (

    <div style={styles.container}>

      {products.map((product) => (

        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
          rating={product.rating}
          image={product.image}
        />

      ))}

    </div>
  );
}

const styles = {

  container: {
    display: "flex",
    gap: "20px",
    flexWrap: "wrap",
    justifyContent: "center"
  },

  card: {
    width: "220px",
    border: "1px solid #ddd",
    borderRadius: "10px",
    padding: "15px",
    textAlign: "center",
    boxShadow: "0 2px 8px rgba(0,0,0,0.2)"
  },
}
```

```
image: {
  width: "100%",
  borderRadius: "10px"
},

button: {
  backgroundColor: "orange",
  border: "none",
  padding: "10px",
  width: "100%",
  cursor: "pointer",
  borderRadius: "5px",
  marginTop: "10px"
}
};

export default App;
```

Concepts Used

- ✓ Props
- ✓ Arrays
- ✓ map()
- ✓ Dynamic Components
- ✓ Dynamic Styling
- ✓ Reusable Components
- ✓ Rendering Lists

Challenge for You

Add:

- Discount
- Product category
- Dark mode
- Hover effect
- Buy Now button
- Search filter

- Rating color change

Interview Questions

Why use map() here?

To dynamically render multiple product cards.

Why use key?

To uniquely identify rendered components.

Why create ProductCard separately?

For reusable and cleaner code.

You said:

ChatGPT said:

Handling Click Events in React

What is a Click Event?

A click event happens when a user clicks on:

- button
- image
- div
- card
- any element

React uses:

onClick

to handle click events.

Basic Syntax

```
JavaScript<button onClick={functionName}>
  Click
</button>
```

First Example

```
JavaScriptfunction App() {

  function handleClick() {
    alert("Button Clicked");
  }

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

Output

Alert appears when button clicked

How It Works

Step 1

Button clicked.

Step 2

onClick event triggers.

Step 3

Function runs.

Important Rule

✗ Wrong

```
JavaScript<button onClick={handleClick()}>
```

This runs immediately.

Correct



```
JavaScript<button onClick={handleClick}>
```

Pass function reference only.

Using Arrow Function

Very Common 🚀

```
JavaScriptfunction App() {  
  
  return (  
    <button  
      onClick={() => alert("Hello")}  
    >  
      Click  
    </button>  
  );  
}
```

Passing Arguments in Click Event

```
JavaScriptfunction App() {  
  
  function greet(name) {  
    alert(`Hello ${name}`);  
  }  
  
  return (  
    <button  
      onClick={() => greet("Aamir")}  
    >  
      Click  
    </button>  
  );  
}
```

Output

Hello Aamir

Updating State on Click

Very Important 🚀

Counter Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() => setCount(count + 1)}
      >
        Increase
      </button>

    </div>
  );
}
```

Output

Count increases on every click

Toggle Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [show, setShow] = useState(false);

  return (
    <div>

      <button
        onClick={() => setShow(!show)}
      >
        Toggle
      </button>

      {show && <h1>Hello React</h1>}

    </div>
  );
}
```

Output

Text show/hide on click

Click Event on Different Elements

Button

```
JavaScript<button onClick={handleClick}>
```

Image

```
JavaScript
```

Div

```
JavaScript<div onClick={handleClick}>  
  Click Area  
</div>
```

Dynamic Click Events with Arrays

Very Common 🚀

```
JavaScriptfunction App() {  
  
  const fruits = [  
    "Apple",  
    "Mango",  
    "Banana"  
  ];  
  
  function showFruit(name) {  
    alert(name);  
  }  
  
  return (  
    <div>  
  
      {fruits.map((fruit, index) => (  
  
        <button  
          key={index}  
          onClick={() => showFruit(fruit)}  
        >  
          {fruit}  
        </button>  
  
      )}}  
  
    </div>  
  );  
}
```

Output

Clicking button shows fruit name

Event Object

React automatically provides event object.

Example

```
JavaScriptfunction App() {  
  
  function handleClick(event) {  
    console.log(event);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click  
    </button>  
  );  
}
```

Common Event Properties

```
event.target  
event.type  
event.clientX  
event.clientY
```

Prevent Default

Used in forms.

```
JavaScriptfunction App() {  
  
  function handleSubmit(event) {  
    event.preventDefault();  
  
    alert("Form Submitted");  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
  
      <button type="submit">  
        Submit  
      </button>  
  
    </form>  
  );  
}
```

Event Bubbling

Events move from child → parent.

Example

```
JavaScript<div onClick={() => alert("Parent")}>  
  
  <button onClick={() => alert("Child")}>  
    Click  
  </button>  
  
</div>
```

Output

```
Child  
Parent
```

Stop Event Bubbling

```
JavaScriptfunction handleClick(event) {  
  
  event.stopPropagation();  
  
  alert("Child");  
}
```

Common Mistakes

1. Calling Function Directly

✗ Wrong

```
JavaScriptonClick={handleClick()}
```

2. Using onclick Instead of onClick

✗ Wrong

```
JavaScriptonclick
```

✓ Correct

```
JavaScriptonClick
```

React uses camelCase.

3. Forgetting Arrow Function for Arguments

✗ Wrong

```
JavaScriptonClick={greet("Aamir")}
```

✓ Correct

```
JavaScriptonClick={() => greet("Aamir")}
```


Real Project Examples

Handling click events used in:

- Add to Cart
- Like button
- Login button
- Dark mode toggle
- Delete item
- Open modal
- Navigation

Interview Questions

Which event is used for clicks in React?

onClick

Why use arrow function in onClick?

To pass arguments without immediately calling function.

What does preventDefault() do?

Stops default browser behavior.

Final Short Notes

Handling Click Events:

- Uses onClick
- Runs function on click
- Can update state
- Can pass arguments
- Uses event object
- preventDefault() stops default action

You said:

Handling non click events

ChatGPT said:

Handling Non-Click Events in React

What are Non-Click Events?

Events other than `onClick` are called non-click events.

Used for:

- typing
- submitting forms
- keyboard actions
- mouse actions
- focus/blur
- loading

Common Non-Click Events in React

Event	Use
<code>onChange</code>	Input change
<code>onSubmit</code>	Form submit
<code>onKeyDown</code>	Key press
<code>onKeyUp</code>	Key release
<code>onMouseOver</code>	Mouse hover
<code>onMouseOut</code>	Mouse leave
<code>onFocus</code>	Input focus
<code>onBlur</code>	Input blur
<code>onDoubleClick</code>	Double click

Event	Use
onScroll	Scroll event

1. onChange Event

Most Important 🚀

Used with:

- input
- textarea
- select

Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [name, setName] = useState("");

  return (
    <div>

      <input
        type="text"
        onChange={(event) =>
          setName(event.target.value)
        }
      />

      <h1>{name}</h1>

    </div>
  );
}
```

Output

Text updates while typing

How It Works

User types
↓
onChange runs
↓
State updates
↓
UI updates

2. onSubmit Event

Used in forms.

Example

```
JavaScriptfunction App() {  
  
  function handleSubmit(event) {  
  
    event.preventDefault();  
  
    alert("Form Submitted");  
  }  
  
  return (  
  
    <form onSubmit={handleSubmit}>  
  
      <button type="submit">  
        Submit  
      </button>  
  
    </form>  
  );  
}
```

Why preventDefault()?

Without it:

Page reloads

3. onKeyDown Event

Runs when key is pressed.

Example

```
JavaScriptfunction App() {  
  
  function handleKey(event) {  
    console.log(event.key);  
  }  
  
  return (  
    <input  
      type="text"  
      onKeyDown={handleKey}  
    />  
  );  
}
```

Output

Pressed key prints in console

4. onKeyUp Event

Runs when key released.

Example

```
JavaScript<input onKeyUp={handleKey} />
```

Difference

Event	Runs
-------	------

onKeyDown	Key pressed
-----------	-------------

Event	Runs
onKeyUp	Key released

5. onMouseOver Event

Runs when mouse enters element.

Example

```
JavaScriptfunction App() {  
  
  return (  
    <h1  
      onMouseOver={() =>  
        alert("Mouse Over")  
      }  
    >  
      Hover Me  
    </h1>  
  );  
}
```

6. onMouseOut Event

Runs when mouse leaves element.

Example

```
JavaScript<h1  
  onMouseOut={() =>  
    alert("Mouse Out")  
  }  
>  
  Hover  
</h1>
```

7. onFocus Event

Runs when input gets focus.

Example

```
JavaScript<input
  onFocus={() =>
    console.log("Focused")
  }
/>
```

8. onBlur Event

Runs when input loses focus.

Example

```
JavaScript<input
  onBlur={() =>
    console.log("Blur")
  }
/>
```

9. onDoubleClick Event

Runs on double click.

Example

```
JavaScript<button
  onDoubleClick={() =>
    alert("Double Clicked")
  }
>
  Double Click
</button>
```

10. onScroll Event

Runs while scrolling.

Example

```
JavaScript<div
  onScroll={() =>
    console.log("Scrolling")
  }
  style={{
    height: "100px",
    overflow: "scroll"
  }}
>

  Long Content

</div>
```

Event Object

React automatically provides event object.

Example

```
JavaScriptfunction handleChange(event) {

  console.log(event.target.value);
}
```

Important Event Properties

Property	Meaning
event.target	Current element
event.target.value	Input value
event.key	Pressed key
event.type	Event type

Real Project Examples

Non-click events used in:

- Forms

- Search bars
- Login pages
- Validation
- Hover effects
- Keyboard shortcuts
- Scroll animations

Common Mistakes

1. Using onchange Instead of onChange

✗ Wrong

```
JavaScriptonchange
```

✓ Correct

```
JavaScriptonChange
```

React uses camelCase.

2. Forgetting preventDefault()

✗ Form reloads page.

3. Not Using event.target.value

✗ Wrong

```
JavaScriptsetName(event)
```

✓ Correct

```
JavaScriptsetName(event.target.value)
```

Best Practices

✓ Use:

- controlled components
- state handling
- reusable handlers

Interview Questions

Which event is used for input changes?

onChange

What does onSubmit do?

Handles form submission.

Difference between onKeyDown and onKeyUp?

Event	Meaning
onKeyDown	Key pressed
onKeyUp	Key released

Why use preventDefault()?

To stop default browser behavior.

Final Short Notes

Non-Click Events:

- onChange
- onSubmit
- onKeyDown
- onKeyUp
- onFocus
- onBlur
- onMouseOver
- onMouseOut
- onScroll

Used for:

- forms
- typing
- keyboard actions
- hover effects

You said:

ChatGPT said:

Event Objects in React

What is an Event Object?

When an event happens in React:

- click
- typing
- submit
- hover

React automatically creates an:

Event Object

This object contains information about the event.

Simple Definition

Event object stores details about an event.

Example

```
JavaScriptfunction App() {  
  
  function handleClick(event) {  
  
    console.log(event);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click  
    </button>  
  );  
}
```

Output

Console shows:

```
SyntheticBaseEvent {...}
```

Why Event Object Important?

Used for:

- getting input value
- detecting pressed key
- mouse position
- preventing form reload

- stopping event bubbling

Event Object Flow

```
graph TD; A[User Action] --> B[Event Happens]; B --> C[React Creates Event Object]; C --> D[Function Receives Event];
```

Common Syntax

```
JavaScriptfunction handleEvent(event) {  
  
}
```

OR

```
JavaScript(event) => {}
```

1. event.target

Very Important 🚀

Returns element where event happened.

Example

```
JavaScriptfunction App() {  
  
  function handleClick(event) {  
  
    console.log(event.target);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click  
    </button>  
  );  
}
```

Output

```
<button>Click</button>
```

2. event.target.value

Most Used 🚀

Used in input fields.

Example

```
JavaScriptfunction App() {  
  
  function handleChange(event) {  
  
    console.log(event.target.value);  
  }  
  
  return (  
    <input  
      type="text"  
      onChange={handleChange}  
    />  
  );  
}
```

Output

Whatever user types

3. event.type

Returns event type.

Example

```
JavaScriptfunction App() {  
  
  function handleClick(event) {  
  
    console.log(event.type);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click  
    </button>  
  );  
}
```

Output

click

4. event.key

Used in keyboard events.

Example

```
JavaScriptfunction App() {  
  
  function handleKey(event) {  
  
    console.log(event.key);  
  }  
  
  return (  
    <input  
      onKeyDown={handleKey}  
    />  
  );  
}
```

Output

```
a  
Enter  
Shift
```

5. event.preventDefault()

Very Important 🚀

Stops default browser behavior.

Example Without preventDefault()

```
JavaScriptfunction App() {  
  
  function handleSubmit() {  
    alert("Submitted");  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
  
      <button type="submit">  
        Submit  
      </button>  
  
    </form>  
  );  
}
```

Problem

Page reloads automatically

Correct Version

```
JavaScriptfunction App() {  
  
  function handleSubmit(event) {  
  
    event.preventDefault();  
  
    alert("Submitted");  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
  
      <button type="submit">  
        Submit  
      </button>  
  
    </form>  
  );  
}
```

6. event.stopPropagation()

Stops event bubbling.

Example

```
JavaScriptfunction App() {  
  
  function parentClick() {  
    alert("Parent");  
  }  
  
  function childClick(event) {  
  
    event.stopPropagation();  
  
    alert("Child");  
  }  
  
  return (  
  
    <div onClick={parentClick}>  
  
      <button onClick={childClick}>  
        Click  
      </button>  
  
    </div>  
  );  
}
```

Output

Only Child alert appears

7. Mouse Position

Example

```
JavaScriptfunction App() {  
  
  function handleMouse(event) {  
  
    console.log(event.clientX);  
    console.log(event.clientY);  
  }  
  
  return (  
    <div onMouseMove={handleMouse}>  
      Move Mouse  
    </div>  
  );  
}
```

Output

Mouse X and Y coordinates

Synthetic Events in React

React uses:

SyntheticEvent

instead of normal browser events.

Why React Uses Synthetic Events?

Because they:

- work same in all browsers
- improve performance

- provide consistency

Common Event Properties

Property	Meaning
event.target	Element that triggered event
event.target.value	Input value
event.type	Event type
event.key	Pressed key
event.clientX	Mouse X position
event.clientY	Mouse Y position

Real Project Examples

Event objects used in:

- Forms
- Search bars
- Login pages
- Validation
- Drag & drop
- Keyboard shortcuts
- Mouse tracking

Common Mistakes

1. Forgetting event Parameter

✗ Wrong

```
JavaScriptfunction handleClick() {  
  console.log(event);  
}
```

Correct



```
JavaScriptfunction handleClick(event) {  
  console.log(event);  
}
```

2. Using preventDefault() Incorrectly

✗ Wrong

```
JavaScriptpreventDefault();
```



Correct

```
JavaScriptevent.preventDefault();
```

3. Using target Instead of target.value

✗ Wrong

```
JavaScriptconsole.log(event.target);
```

when input value needed.

Correct

```
JavaScriptconsole.log(event.target.value);
```

Interview Questions

What is an event object?

An object containing information about an event.

What does event.target do?

Returns the element where event happened.

What does preventDefault() do?

Stops default browser behavior.

What does stopPropagation() do?

Stops event bubbling.

Final Short Notes

Event Object:

- Automatically created by React
- Stores event information

Important Methods:

- event.target
- event.target.value
- event.key
- event.preventDefault()
- event.stopPropagation()

You said:

ChatGPT said:

State in React

What is State?

State is used to store:

- changing data
- dynamic values
- interactive UI data

Simple Definition

State is data that can change over time.

Real Life Example

Examples of changing data:

- counter value
- dark mode
- form input
- like button
- todo list

These are stored in:

State

Why State is Important?

State makes React apps:

- dynamic
- interactive
- responsive

Without state:

UI cannot update dynamically

State vs Props

Props	State
Passed from parent	Managed inside component
Read only	Mutable
External data	Internal data

useState Hook

React uses:

```
useState()
```

to create state.

Syntax

```
JavaScriptconst [state, setState] = useState(initialValue);
```

Understanding Syntax

```
JavaScriptconst [count, setCount] = useState(0);
```

count

Current state value.

setCount

Function to update state.

0

Initial value.

First State Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

Output

Count increases when button clicked

How State Works

```
Button Click
↓
setCount() runs
↓
State updates
↓
Component re-renders
↓
UI updates
```

Why UI Updates Automatically?

Because React:

Re-renders component when state changes

Multiple State Variables

```
JavaScriptimport { useState } from "react";

function App() {

  const [name, setName] = useState("Aamir");

  const [age, setAge] = useState(21);

  return (
    <div>

      <h1>{name}</h1>
      <h2>{age}</h2>

    </div>
  );
}
```

Updating State

Example

```
JavaScriptsetCount(count + 1);
```

Another Way (Best Practice)

Very Important 🚀

```
JavaScriptsetCount((prev) => prev + 1);
```

Why Better?

Because it uses:

latest previous state value

Toggle State Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [dark, setDark] = useState(false);

  return (

    <div>

      <button
        onClick={() =>
          setDark(!dark)
        }
      >
        Toggle
      </button>

      <h1>
        {dark ? "Dark" : "Light"}
      </h1>

    </div>
  );
}
```

Output

Dark ↔ Light

State with Input Field

Very Important 🚀

```
JavaScriptimport { useState } from "react";

function App() {

  const [name, setName] = useState("");

  return (
    <div>

      <input
        type="text"
        onChange={(e) =>
          setName(e.target.value)
        }
      />

      <h1>{name}</h1>

    </div>
  );
}
```

Output

Text updates while typing

State with Arrays

```
JavaScriptimport { useState } from "react";

function App() {

  const [fruits, setFruits] =
    useState(["Apple"]);

  function addFruit() {

    setFruits([
      ...fruits,
      "Mango"
    ]);
  }

  return (
    <div>

      <button onClick={addFruit}>
        Add Fruit
      </button>

      {fruits.map((fruit, index) => (
        <h1 key={index}>{fruit}</h1>
      ))}

    </div>
  );
}
```

Output

```
Apple
Mango
```

State with Objects

```
JavaScriptimport { useState } from "react";

function App() {

  const [user, setUser] = useState({
    name: "Aamir",
    age: 21
  });

  return (
    <div>

      <h1>{user.name}</h1>

      <button
        onClick={() =>
          setUser({
            ...user,
            age: 22
          })
        }
      >
        Change Age
      </button>

    </div>
  );
}
```

Important Rule

Never update state directly.

Wrong

```
JavaScriptcount = count + 1;
```

Correct

```
JavaScriptsetCount(count + 1);
```

Why?

Because React only tracks updates using:

```
setState function
```

Component Re-rendering

Whenever state changes:

```
Component renders again
```

This is called:

```
Re-rendering
```

Common Mistakes

1. Direct State Update

✗ Wrong

```
JavaScriptcount++;
```

2. Forgetting useState Import

✗ Wrong

```
JavaScriptconst [count, setCount] = useState(0);
```

without:

```
JavaScriptimport { useState } from "react";
```

3. Mutating Arrays/Objects

✗ Wrong

```
JavaScriptfruits.push("Mango");
```

✓ Correct

```
JavaScriptsetFruits([...fruits, "Mango"]);
```

Real Project Examples

State used in:

- Counters
- Forms
- Login systems
- Todo apps
- Dark mode
- Shopping cart
- Search bar

Interview Questions

What is state in React?

State is data that changes over time and updates the UI.

Which hook is used for state?

```
useState()
```

Does state update UI automatically?

Yes.

Can we change state directly?

No.

Final Short Notes

State:

- Stores changing data
- Makes UI dynamic
- Created using useState()

Syntax:

```
const [state, setState] =  
useState(initialValue)
```

Important:

- Never change state directly
- Use setter function
- State change causes re-render

You said:

state in react for interview

ChatGPT said:

React State Interview Questions & Answers

1. What is State in React?

State is a built-in object used to store dynamic and changing data in a component.

State changes cause component re-rendering.

2. Why is State Used?

State is used to:

- make UI dynamic
- handle user interaction

- store changing values

Examples:

- counter
- form data
- dark mode
- todo list

3. Which Hook is Used for State?

React uses:

```
useState()
```

hook.

4. Syntax of useState()

```
JavaScriptconst [state, setState] = useState(initialValue);
```

5. Explain useState()

Example:

```
JavaScriptconst [count, setCount] = useState(0);
```

Part	Meaning
count	Current state
setCount	Function to update state
0	Initial value

6. What Happens When State Changes?

```
Component re-renders automatically.
```

React updates UI with new state.

7. Can We Update State Directly?

✗ No

Wrong:

```
JavaScriptcount = count + 1;
```

Correct:

```
JavaScriptsetCount(count + 1);
```

8. Why Should We Not Update State Directly?

Because React tracks updates only through:

```
setter function
```

Direct updates do not trigger re-rendering.

9. What is Re-rendering?

```
Re-rendering means React runs component again when state changes.
```

10. Example of State

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

11. Difference Between Props and State

Props	State
Passed from parent	Managed inside component
Read-only	Mutable
External data	Internal data

12. Can State Store Arrays?

✓ Yes

```
JavaScriptconst [fruits, setFruits] =
  useState(["Apple"]);
```

13. Can State Store Objects?

✓ Yes

```
JavaScriptconst [user, setUser] = useState({  
  name: "Aamir"  
});
```

14. Updating Array State

```
JavaScriptsetFruits([...fruits, "Mango"]);
```

15. Updating Object State

```
JavaScriptsetUser({  
  ...user,  
  age: 22  
});
```

16. What is Previous State?

React provides previous state value.

Example

```
JavaScriptsetCount((prev) => prev + 1);
```

Why Preferred?

Because it always uses latest state value.

17. Controlled Component

Input controlled by React state.

Example

```
JavaScriptconst [name, setName] = useState("");

<input
  value={name}
  onChange={(e) =>
    setName(e.target.value)
  }
/>
```

18. What is Initial State?

Default value passed to:

```
useState()
```

Example:

```
JavaScriptuseState(0)
```

0 is initial state.

19. Is State Asynchronous?

✓ Yes

React state updates are asynchronous.

Example

```
JavaScriptsetCount(count + 1);
console.log(count);
```

Old value may print immediately.

20. Can We Use Multiple States?

✓ Yes

```
JavaScriptconst [name, setName] = useState("");  
const [age, setAge] = useState(21);
```

21. What is Lifting State Up?

Sharing state between components by moving state to parent component.

22. What Happens if State Changes Frequently?

Frequent state updates cause:

multiple re-renders

May affect performance.

23. Functional State Update

Best Practice 🚀

```
JavaScriptsetCount((prev) => prev + 1);
```

24. Common State Mistakes

Direct Mutation ❌

```
JavaScriptuser.name = "Ali";
```

Correct ✅

```
JavaScriptsetUser({  
  ...user,  
  name: "Ali"  
});
```

25. Real Project Uses of State

Used in:

- Login forms

- Shopping cart
- Search bar
- Theme toggle
- Todo apps
- API data handling

Most Asked Interview Questions

Q1. What is state in React?

State is an object that stores dynamic data and updates UI when changed.

Q2. Which hook manages state?

```
useState()
```

Q3. Does changing state re-render component?

✓ Yes.

Q4. Can we update state directly?

✗ No.

Q5. Difference between props and state?

Props are external/read-only. State is internal/mutable.

Q6. Why use functional update?

To safely access latest previous state.

Q7. Is useState synchronous?

✗ No, asynchronous.

Quick Revision Notes

State:

- Stores changing data
- Managed using useState()
- State updates cause re-render
- Never update directly
- Can store:
 - string
 - number
 - array
 - object
- use setter function always

You said:

ChatGPT said:

React Hooks

What are Hooks?

Hooks are special React functions that let us use:

- state
- lifecycle features
- React features

inside:

Functional Components

Simple Definition

Hooks allow functional components to use React features.

Before Hooks

Earlier:

- Functional components → simple UI only
- Class components → state & lifecycle

After Hooks:

✅ Functional components can also use:

- state
- effects
- context
- refs

Why Hooks Important?

Hooks make code:

- simpler
- cleaner
- reusable
- easier to manage

Rules of Hooks

1. Hooks start with use

Examples:

```
useState  
useEffect  
useRef  
useContext
```

2. Call Hooks at Top Level

❌ Wrong

```
JavaScriptif(true){  
  useState()  
}
```

✓ Correct

```
JavaScriptuseState()
```

3. Use Hooks Inside Functional Components

✗ Wrong

```
JavaScriptfunction normalFunction() {  
  useState()  
}
```

Most Important React Hooks

Hook	Purpose
useState	Manage state
useEffect	Side effects
useRef	Access DOM / persist values
useContext	Global data
useReducer	Complex state
useMemo	Performance optimization
useCallback	Function optimization

1. useState Hook

Most Important 

Used for:

Managing state

Syntax

```
JavaScriptconst [state, setState] =  
  useState(initialValue);
```

Example

```
JavaScriptimport { useState } from "react";  
  
function App() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return (  
    <div>  
  
      <h1>{count}</h1>  
  
      <button  
        onClick={() =>  
          setCount(count + 1)  
        }  
      >  
        Increase  
      </button>  
  
    </div>  
  );  
}
```

2. useEffect Hook

Very Important 🚀

Used for:

- API calls
- timers
- side effects

- lifecycle handling

Syntax

```
JavaScriptuseEffect(() => {  
  
}, []);
```

Example

```
JavaScriptimport { useEffect } from "react";  
  
function App() {  
  
  useEffect(() => {  
  
    console.log("Component Mounted");  
  
  }, []);  
  
  return <h1>Hello</h1>;  
}
```

Dependency Array

Dependency	Meaning
------------	---------

[]	Run once
----	----------

[count]	Run when count changes
---------	------------------------

no array	Run every render
----------	------------------

3. useRef Hook

Used for:

- accessing DOM

- storing values without re-render

Example

```
JavaScriptimport { useRef } from "react";

function App() {

  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>

      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>

    </div>
  );
}
```

4. useContext Hook

Used for:

Global state management

Avoids:

Props drilling

Example

```
JavaScriptconst UserContext = createContext();
```

5. useReducer Hook

Used for:

- complex state logic
- multiple state updates

Syntax

```
JavaScriptconst [state, dispatch] =  
  useReducer(reducer, initialState);
```

Example

```
JavaScriptfunction reducer(state, action) {  
  
  switch(action.type){  
  
    case "increment":  
      return { count: state.count + 1 };  
  
    default:  
      return state;  
  }  
}
```

Why useReducer?

Better for:

- large apps
- complex state
- predictable logic

6. useMemo Hook

Performance optimization hook 🚀

Used to:

```
Memoize expensive calculations
```

Example

```
JavaScriptconst result = useMemo(() => {  
  
  return expensiveCalculation();  
  
}, []);
```

Why useMemo?

Prevents unnecessary recalculation.

7. useCallback Hook

Used to:

Memoize functions

Example

```
JavaScriptconst handleClick = useCallback(() => {  
  
  console.log("Clicked");  
  
}, []);
```

Why useCallback?

Prevents unnecessary function recreation.

Difference Between useMemo & useCallback

Hook	Memoizes
useMemo	Value
useCallback	Function

Custom Hooks

We can create our own hooks.

Example

```
JavaScriptfunction useCounter() {  
  
  const [count, setCount] =  
    useState(0);  
  
  function increment() {  
    setCount(count + 1);  
  }  
  
  return { count, increment };  
}
```

Benefits of Hooks

✅ Cleaner code ✅ Reusable logic ✅ Less boilerplate ✅ Easier maintenance ✅ Better readability

Common Mistakes

1. Calling Hooks Inside Condition

❌ Wrong

```
JavaScriptif(show){  
  useEffect()  
}
```

2. Forgetting Dependency Array

May cause:

Infinite re-renders

3. Mutating State Directly

❌ Wrong

```
JavaScriptcount++;
```

Real Project Usage

Hooks used in:

- API calls
- forms
- dark mode
- authentication
- search
- optimization
- global state

Most Asked Interview Questions

What are Hooks?

Hooks are functions that allow functional components to use React features.

Which hook manages state?

useState

Which hook handles side effects?

useEffect

Which hook accesses DOM?

useRef

Difference between useMemo and useCallback?

useMemo

useCallback

Memoizes value Memoizes function

Why useContext?

To avoid props drilling.

Final Revision Notes

Important Hooks:

- useState
- useEffect
- useRef
- useContext
- useReducer
- useMemo
- useCallback

Hooks:

- work in functional components
- simplify React development
- improve reusability

You said:

useState useEffect useRef useContext

ChatGPT said:

Important React Hooks

1. useState Hook

What is useState?

useState is used to:

Store and manage changing data in a component.

Syntax

```
JavaScriptconst [state, setState] =  
  useState(initialValue);
```

Example

```
JavaScriptimport { useState } from "react";  
  
function App() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return (  
    <div>  
  
      <h1>{count}</h1>  
  
      <button  
        onClick={() =>  
          setCount(count + 1)  
        }  
      >  
        Increase  
      </button>  
  
    </div>  
  );  
}
```

Output

Count increases on button click

Why useState Important?

Used for:

- counters
- forms
- dark mode

- todo apps
- dynamic UI

Key Points

✓ State updates cause re-render ✓ Never update state directly ✓ Use setter function

2. useEffect Hook

What is useEffect?

Used for:

Handling side effects

What are Side Effects?

Examples:

- API calls
- timers
- local storage
- event listeners

Syntax

```
JavaScriptuseEffect(() => {  
  
}, []);
```

Example

```
JavaScriptimport { useEffect } from "react";

function App() {

  useEffect(() => {

    console.log("Component Mounted");

  }, []);

  return <h1>Hello</h1>;
}
```

Dependency Array

Dependency	Meaning
------------	---------

[]	Run once
----	----------

[count]	Run when count changes
---------	------------------------

Dependency Meaning

no array Run every render

Example with State

```
JavaScriptimport { useState, useEffect }
from "react";

function App() {

  const [count, setCount] =
    useState(0);

  useEffect(() => {

    console.log("Count Changed");

  }, [count]);

  return (
    <button
      onClick={() =>
        setCount(count + 1)
      }
    >
      {count}
    </button>
  );
}
```

Why useEffect Important?

Used for:

- fetching API data
- timers
- authentication

- lifecycle handling

3. useRef Hook

What is useRef?

Used for:

- accessing DOM elements
- storing values without re-render

Syntax

```
JavaScriptconst refName = useRef();
```

Example

```
JavaScriptimport { useRef } from "react";

function App() {

  const inputRef = useRef();

  function focusInput() {

    inputRef.current.focus();
  }

  return (
    <div>

      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>

    </div>
  );
}
```


Output

Input gets focused

Important Point

Changing **useRef** value:

Does NOT re-render component

useRef vs useState

useState

useRef

Causes re-render

No re-render

Stores dynamic UI data Stores mutable values

Real Uses of useRef

- focus input
- scroll position
- timers
- previous values
- DOM access

4. useContext Hook

What is useContext?

Used for:

Sharing global data between components.

Avoids:

Props drilling

What is Props Drilling?

Passing props through many components.

Flow of useContext

```
Create Context
↓
Provide Context
↓
Consume Context
```

Step 1: Create Context

```
JavaScriptimport { createContext } from "react";

const UserContext =
  createContext();
```

Step 2: Provide Context

```
JavaScript<UserContext.Provider value="Aamir">

  <Child />

</UserContext.Provider>
```

Step 3: Use Context

```
JavaScriptimport { useContext } from "react";

function Child() {

  const user =
    useContext(UserContext);

  return <h1>{user}</h1>;
}
```

Complete Example

```
JavaScriptimport {
  createContext,
  useContext
} from "react";

const UserContext =
  createContext();

function Child() {

  const user =
    useContext(UserContext);

  return <h1>{user}</h1>;
}

function App() {

  return (

    <UserContext.Provider
      value="Aamir"
    >

      <Child />

    </UserContext.Provider>

  );
}
```

Output

Aamir

Why useContext Important?

Used for:

- authentication
- theme
- language

- user data
- global state

Quick Comparison

Hook	Purpose
useState	Manage state
useEffect	Handle side effects
useRef	Access DOM/store mutable value
useContext	Global data sharing

Common Interview Questions

Which hook manages state?

useState

Which hook handles side effects?

useEffect

Which hook accesses DOM?

useRef

Which hook avoids props drilling?

useContext

Does useRef cause re-render?

 No.

What is dependency array in useEffect?

Controls when effect runs.

Final Revision Notes

useState:
- manages changing data

useEffect:
- handles side effects

useRef:
- accesses DOM
- no re-render

useContext:
- shares global data
- avoids props drilling

You said:

ChatGPT said:

useState Hook in React

What is useState?

`useState` is a React Hook used to:

Store and manage changing data in functional components.

Simple Definition

useState allows components to remember and update values.

Why useState is Needed?

Without `useState`:

```
UI cannot update dynamically.
```

React apps become static.

Real Life Examples

State used in:

- counter
- dark mode
- forms
- search bar
- todo list
- like button

Import useState

```
JavaScriptimport { useState } from "react";
```

Syntax

```
JavaScriptconst [state, setState] =  
  useState(initialValue);
```

Understanding Syntax

```
JavaScriptconst [count, setCount] =  
  useState(0);
```

Part	Meaning
------	---------

count	Current state value
-------	---------------------

setCount	Function to update state
----------	--------------------------

Part	Meaning
0	Initial value

First Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] =
    useState(0);

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

Output

0 → 1 → 2 → 3

How useState Works

Button Click
↓
setCount() runs
↓
State updates
↓
Component re-renders
↓
UI updates automatically

Why UI Updates Automatically?

Because React:

Re-renders component when state changes.

Updating State

Example

```
JavaScriptsetCount(count + 1);
```

Functional Update (Best Practice) 🚀

```
JavaScriptsetCount((prev) => prev + 1);
```

Why Better?

Uses latest previous state value.

Especially useful for:

- multiple updates

- async updates

Multiple State Variables

```
JavaScriptimport { useState } from "react";

function App() {

  const [name, setName] =
    useState("Aamir");

  const [age, setAge] =
    useState(21);

  return (
    <div>

      <h1>{name}</h1>
      <h2>{age}</h2>

    </div>
  );
}
```

useState with Input Field

Very Important 🚀

```
JavaScriptimport { useState } from "react";

function App() {

  const [name, setName] =
    useState("");

  return (
    <div>

      <input
        type="text"
        value={name}
        onChange={(e) =>
          setName(e.target.value)
        }
      />

      <h1>{name}</h1>

    </div>
  );
}
```

Output

Text updates while typing

useState with Boolean

Used for:

- dark mode
- toggle

- show/hide

Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [show, setShow] =
    useState(false);

  return (
    <div>

      <button
        onClick={() =>
          setShow(!show)
        }
      >
        Toggle
      </button>

      {show && <h1>Hello</h1>}

    </div>
  );
}
```

useState with Arrays

```
JavaScriptimport { useState } from "react";

function App() {

  const [fruits, setFruits] =
    useState(["Apple"]);

  function addFruit() {

    setFruits([
      ...fruits,
      "Mango"
    ]);
  }

  return (
    <div>

      <button onClick={addFruit}>
        Add Fruit
      </button>

      {fruits.map((fruit, index) => (
        <h1 key={index}>{fruit}</h1>
      ))}

    </div>
  );
}
```

Output

Apple
Mango

useState with Objects

```
JavaScriptimport { useState } from "react";

function App() {

  const [user, setUser] =
    useState({
      name: "Aamir",
      age: 21
    });

  return (
    <div>

      <h1>{user.name}</h1>

      <button
        onClick={() =>
          setUser({
            ...user,
            age: 22
          })
        }
      >
        Change Age
      </button>

    </div>
  );
}
```

Important Rule

Never update state directly.

Wrong

```
JavaScriptcount++;
```

OR

```
JavaScriptuser.name = "Ali";
```

Correct 

```
JavaScriptsetCount(count + 1);
```

Why?

Because React tracks updates through:

```
setter function
```

Component Re-rendering

Whenever state changes:

```
Component renders again.
```

Called:

```
Re-rendering
```

Common Mistakes

1. Forgetting useState Import

 Wrong

```
JavaScriptconst [count, setCount] =  
  useState(0);
```

without:

```
JavaScriptimport { useState } from "react";
```

2. Direct State Mutation

 Wrong

```
JavaScriptfruits.push("Mango");
```

Correct

```
JavaScriptsetFruits([...fruits, "Mango"]);
```

3. Using Old State Value

✗ Sometimes problematic

```
JavaScriptsetCount(count + 1);  
setCount(count + 1);
```

Better



```
JavaScriptsetCount((prev) => prev + 1);  
setCount((prev) => prev + 1);
```

Real Project Uses

useState used in:

- login forms
- search bars
- shopping cart
- dark mode
- counters
- API data display

Most Asked Interview Questions

What is useState?

A hook used to manage state in functional components.

Does useState cause re-render?

✓ Yes.

Can state be array/object?

✓ Yes.

Can we update state directly?

✗ No.

Why use functional update?

To safely use latest previous state.

Final Revision Notes

useState:

- manages dynamic data
- updates UI automatically
- uses setter function

Syntax:

```
const [state, setState] =  
  useState(initialValue)
```

Important:

- never mutate state directly
- state updates cause re-render

You said:

ChatGPT said:

useEffect Hook in React

What is useEffect?

useEffect is a React Hook used to:

Handle side effects in functional components.

Simple Definition

`useEffect` runs code after component renders.

What are Side Effects?

Side effects mean operations outside normal UI rendering.

Examples:

- API calls
- timers
- local storage
- event listeners
- data fetching

Why `useEffect` Important?

Without `useEffect`:

Side effect logic becomes difficult to manage.

Import `useEffect`

```
JavaScriptimport { useEffect } from "react";
```

Basic Syntax

```
JavaScriptuseEffect(() => {  
  
  // code here  
  
}, []);
```

Structure of useEffect

```
useEffect(  
  function,  
  dependency array  
)
```

First Example

```
JavaScriptimport { useEffect } from "react";  
  
function App() {  
  
  useEffect(() => {  
  
    console.log("Component Rendered");  
  
  }, []);  
  
  return <h1>Hello React</h1>;  
}
```

Output

Runs once after component mounts

Dependency Array

Very Important 

1. Empty Dependency Array

```
JavaScriptuseEffect(() => {  
  
  console.log("Run Once");  
  
}, []);
```

Meaning

Runs only once when component mounts.

2. Dependency Present

```
JavaScriptuseEffect(() => {  
  
  console.log("Count Changed");  
  
}, [count]);
```

Meaning

Runs whenever count changes.

3. No Dependency Array

```
JavaScriptuseEffect(() => {  
  
  console.log("Every Render");  
  
});
```

Meaning

Runs after every render.

useEffect with useState

Very Common 🚀

```
JavaScriptimport {
  useState,
  useEffect
} from "react";

function App() {

  const [count, setCount] =
    useState(0);

  useEffect(() => {

    console.log("Count Updated");

  }, [count]);

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

Output

Effect runs every time count changes

useEffect for API Calls

Very Important 🚀

```
JavaScriptimport {
  useEffect,
  useState
} from "react";

function App() {

  const [data, setData] =
    useState([]);

  useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => setData(data));

  }, []);

  return (
    <div>

      {data.map((item) => (
        <h1 key={item.id}>
          {item.title}
        </h1>
      ))}

    </div>
  );
}
```

Why [] Used Here?

Because API should load:

Only once when component mounts

Cleanup Function

Very Important 🚀

Used to clean:

- timers
- subscriptions

- event listeners

Syntax

```
JavaScriptuseEffect(() => {  
  
  return () => {  
  
    // cleanup code  
  
  };  
  
}, []);
```

Timer Example

```
JavaScriptimport { useEffect } from "react";  
  
function App() {  
  
  useEffect(() => {  
  
    const timer =  
      setInterval(() => {  
  
        console.log("Running");  
  
      }, 1000);  
  
    return () => {  
  
      clearInterval(timer);  
  
    };  
  
  }, []);  
  
  return <h1>Timer</h1>;  
}
```

Why Cleanup Important?

Prevents:

Memory leaks

Component Lifecycle with useEffect

Lifecycle useEffect

Mount Runs

Update Runs if dependency changes

Unmount Cleanup runs

Common Real Uses

useEffect used for:

- API calls
- authentication
- timers
- fetching data
- event listeners
- local storage

Common Mistakes

1. Infinite Loop

✗ Wrong

```
JavaScriptuseEffect(() => {  
  
  setCount(count + 1);  
  
});
```

Runs forever.

2. Missing Dependency

✗ Wrong

```
JavaScriptuseEffect(() => {  
  
  console.log(count);  
  
}, []);
```

count missing from dependency.

Correct

```
JavaScriptuseEffect(() => {  
  
  console.log(count);  
  
}, [count]);
```

3. Forgetting Cleanup

Can cause:

Memory leaks

useEffect Execution Flow

```
Component Render  
↓  
useEffect Runs  
↓  
Dependency Changes  
↓  
Effect Runs Again  
↓  
Cleanup Runs Before Next Effect
```


Interview Questions

What is useEffect?

A hook used to handle side effects in functional components.

What are side effects?

Operations like:

- API calls
- timers
- local storage
- subscriptions

What does [] mean in useEffect?

Run only once after component mounts.

What happens without dependency array?

Effect runs after every render.

Why use cleanup function?

To prevent memory leaks.

Which hook handles API calls?

```
useEffect
```

Final Revision Notes

useEffect:

- handles side effects
- runs after render

Dependency Array:

- [] → run once
- [count] → run on count change
- no array → every render

Used for:

- API calls
- timers
- event listeners
- local storage

You said:

Activity create likebutton

ChatGPT said:

React Activity: Create Like Button ❤️

Goal

Create a Like Button using:

- useState
- onClick event
- conditional rendering
- dynamic styling

Final Output Idea

💜 Like
Click
❤️ Liked

Step 1: Import useState

```
JavaScriptimport { useState } from "react";
```

Step 2: Create State

```
JavaScriptconst [liked, setLiked] =  
  useState(false);
```

Meaning

Part	Meaning
liked	Current state
setLiked	Updates state
false	Initial value

Step 3: Create Button

```
JavaScript<button>  
  Like  
</button>
```

Step 4: Handle Click Event

```
JavaScriptonClick={() =>  
  setLiked(!liked)  
}
```

How It Works

```
false → true  
true → false
```

Button toggles state.

Complete Code

```
JavaScriptimport { useState } from "react";

function App() {

  const [liked, setLiked] =
    useState(false);

  return (

    <div style={styles.container}>

      <button
        onClick={() =>
          setLiked(!liked)
        }
        style={{
          ...styles.button,

          backgroundColor:
            liked ? "red" : "lightgray",

          color:
            liked ? "white" : "black"
        }}
      >

        {liked ? "❤️ Liked" : "💜 Like"}

      </button>

    </div>
  );
}

const styles = {

  container: {
    display: "flex",
    justifyContent: "center",
    alignItems: "center",
    height: "100vh"
  },

  button: {
    padding: "15px 25px",
    border: "none",
    borderRadius: "10px",
    fontSize: "20px",
```

```
    cursor: "pointer"
  }
};

export default App;
```

Output

Initial:

💜 Like

After Click:

❤️ Liked

Concepts Used

- ✓ useState
- ✓ onClick
- ✓ Conditional Rendering
- ✓ Dynamic Styling
- ✓ State Update

How This Activity Works

Button Click
↓
setLiked(!liked)
↓
State changes
↓
Component re-renders
↓
Text + color update

Challenge for You

Add:

- Like counter
- Animation
- Double click like

- Dark mode
- Hover effect
- Save button
- Heart icon animation

Bonus Activity: Like Counter

```
JavaScriptimport { useState } from "react";

function App() {

  const [likes, setLikes] =
    useState(0);

  return (
    <div>

      <h1>  {likes}</h1>

      <button
        onClick={() =>
          setLikes(likes + 1)
        }
      >
        Like
      </button>

    </div>
  );
}
```

Interview Questions

Which hook used here?

useState

Why UI updates automatically?

Because state change causes re-render.

Why use conditional rendering?

To show different UI based on state.

Final Revision Notes

Like Button:

- uses useState
- uses onClick
- toggles state
- updates UI dynamically

You said:

ChatGPT said:

Toggle Meaning in React

What does Toggle Mean?

Toggle means switching between two states.

Simple Examples

ON ↔ OFF
SHOW ↔ HIDE
DARK ↔ LIGHT
TRUE ↔ FALSE

In React

Toggle usually means:

Changing state from true to false
or false to true

Basic Toggle Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [show, setShow] =
    useState(false);

  return (

    <div>

      <button
        onClick={() =>
          setShow(!show)
        }
      >
        Toggle
      </button>

      {show && <h1>Hello</h1>}

    </div>
  );
}
```

How It Works

Initially:

```
show = false
```

After click:

```
show = true
```

Next click:

```
show = false
```


Important Part

JavaScript!show

! means:

NOT operator

Meaning

Value !Value

true false

false true

Toggle Flow

Click
↓
State changes
↓
true ↔ false
↓
UI updates

Real Project Uses

Toggle used in:

- Dark mode
- Sidebar open/close
- Password show/hide
- Like button
- Dropdown menu

- Navbar menu

Dark Mode Toggle Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [dark, setDark] =
    useState(false);

  return (

    <div>

      <button
        onClick={() =>
          setDark(!dark)
        }
      >
        Toggle Theme
      </button>

      <h1>
        {dark ? "Dark Mode" : "Light Mode"}
      </h1>

    </div>
  );
}
```

Output

Dark Mode ↔ Light Mode

Interview Question

What does toggle mean?

Switching between two states.

Final Short Notes

Toggle:

- changes state between two values
- mostly used with boolean state
- true ↔ false
- uses ! operator

You said:

ChatGPT said:

Closure in JavaScript

What is Closure?

A closure is a function that remembers variables from its outer function even after the outer function finishes.

Simple Definition

Inner function can access outer function variables.

Basic Example

```
JavaScriptfunction outer() {  
  
  let count = 0;  
  
  function inner() {  
  
    count++;  
  
    console.log(count);  
  }  
  
  return inner;  
}  
  
const counter = outer();  
  
counter();  
counter();  
counter();
```

Output

```
1  
2  
3
```

How It Works

Step 1

`outer()` runs.

```
JavaScriptlet count = 0;
```

created.

Step 2

`inner()` returned.

```
JavaScriptreturn inner;
```

Step 3

Even after `outer()` finishes,

`inner()` still remembers:

```
JavaScriptcount
```

This memory is called:

```
Closure
```

Important Point

Closure keeps variables alive.

Another Example

```
JavaScriptfunction greet(name) {  
  
  return function() {  
  
    console.log("Hello " + name);  
  };  
}  
  
const user = greet("Aamir");  
  
user();
```

Output

```
Hello Aamir
```

Why?

Because inner function remembers:

JavaScriptname

Real Life Analogy

Think of closure like:

A backpack

Inner function carries outer variables in its backpack.

Closure Scope Chain

Inner Function
↓
Outer Function Variables
↓
Global Variables

Where Closures are Used?

Closures used in:

- data hiding
- private variables
- callbacks
- event handlers
- timers
- React hooks

- currying

Data Privacy Example

```
JavaScriptfunction bankAccount() {  
  
  let balance = 1000;  
  
  return {  
  
    deposit(amount) {  
  
      balance += amount;  
  
      console.log(balance);  
    },  
  
    getBalance() {  
  
      console.log(balance);  
    }  
  };  
}  
  
const account = bankAccount();  
  
account.deposit(500);  
  
account.getBalance();
```

Output

```
1500  
1500
```

Why Powerful?

Because:

```
balance cannot be accessed directly
```

Private variable created.

Closure with setTimeout

Very Common Interview Question 🚀

Example

```
JavaScriptfunction timer() {  
  
  let message = "Hello";  
  
  setTimeout(function() {  
  
    console.log(message);  
  
  }, 2000);  
}  
  
timer();
```

Output After 2 Seconds

Hello

Why?

Because callback function remembers:

JavaScriptmessage

Common Interview Question 🚀

Problem with var

```
JavaScriptfor(var i = 1; i <= 3; i++) {  
  
  setTimeout(function() {  
  
    console.log(i);  
  
  }, 1000);  
}
```


Output

```
4
4
4
```

Why?

Because:

```
var has function scope
```

All functions share same **i**.

Correct Version Using let

```
JavaScriptfor(let i = 1; i <= 3; i++) {  
  
  setTimeout(function() {  
  
    console.log(i);  
  
  }, 1000);  
}
```

Output

```
1
2
3
```

Why?

Because:

```
let has block scope
```

Each loop gets separate variable.

Closure in React

Closures heavily used in:

- useState
- event handlers
- hooks
- callbacks

Advantages of Closures

✅ Data privacy ✅ State preservation ✅ Better memory management ✅
Powerful async handling

Disadvantages

❌ Extra memory usage possible ❌ Can cause memory leaks if misused

Most Asked Interview Questions

What is closure?

A function remembering variables from outer scope.

Why closures used?

For:

- private variables
- callbacks
- state preservation

Can closure access global variables?

✅ Yes.

Do closures preserve memory?

✅ Yes.

Difference between normal function and closure?

Closure remembers outer variables after outer function ends.

Final Revision Notes

Closure:

- inner function remembers outer variables
- preserves state
- used in callbacks and React

Important Uses:

- private variables
- async operations
- event handlers
- hooks

You said:

Re-render How does it work

ChatGPT said:

Re-render in React

What is Re-render?

Re-render means React runs the component again to update the UI.

Simple Definition

When data changes, React updates the screen.

Why Re-render Happens?

React re-renders when:

- state changes
- props change
- parent component re-renders

- context changes

Most Common Reason

State update

Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] =
    useState(0);

  console.log("Component Rendered");

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

What Happens?

Initially:

Component Rendered

printed once.

After Button Click

State changes
↓
Component function runs again
↓
UI updates

Re-render Flow

User Action
↓
State/Props Change
↓
React Re-renders Component
↓
Virtual DOM Updated
↓
Real DOM Updated Efficiently

Important Point

React does NOT reload page.

It only updates changed parts.

Example with Props

```
JavaScriptfunction Child({ value }) {  
  
  console.log("Child Rendered");  
  
  return <h1>{value}</h1>;  
}
```

If parent changes prop:

Child re-renders

Parent Re-render Example

```
JavaScriptfunction Parent() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return (  
    <Child value={count} />  
  );  
}
```

When parent re-renders:

Child may also re-render

Re-render Does NOT Mean Full Page Reload

✗ Wrong Thinking

Entire website reloads

Correct

✓ React updates only changed UI parts.

Virtual DOM Role

React uses:

Virtual DOM

How It Works

```
Old Virtual DOM  
↓  
New Virtual DOM  
↓  
Compare Differences  
↓  
Update Only Changed Elements
```

This process called:

Diffing

Example

```
JavaScript<h1>{count}</h1>
```

Only this text updates.

Not whole page.

Re-render with useEffect

```
JavaScriptuseEffect(() => {  
  console.log("Effect Runs");  
});
```

Without dependency array:

Runs after every render

Re-render vs Reload

Re-render	Reload
Component updates	Whole page reloads
Fast	Slow
React handles	Browser handles

When React Avoids Re-render?

Using optimization hooks:

- React.memo
- useMemo

- useCallback

Example with React.memo

```
JavaScriptexport default React.memo(Component);
```

Prevents unnecessary re-renders.

Common Causes of Re-render

Cause	Re-render?
State change	✓
Props change	✓
Parent render	✓
Variable change	✗

Important Example

```
JavaScriptlet count = 0;
```

Changing this:

```
Does NOT re-render component
```

Because React only tracks:

- state
- props

Common Mistakes

1. Thinking Normal Variables Trigger UI Update

✗ Wrong


```
JavaScriptlet count = 0;  
count++;
```

UI won't update.

Correct

✓ Use state.

```
JavaScriptconst [count, setCount] =  
  useState(0);
```

2. Too Many Re-renders

✗ Wrong

```
JavaScriptsetCount(count + 1);
```

directly inside component body.

Can create:

```
Infinite re-render
```

Interview Questions

What is re-render in React?

Re-running component to update UI.

What causes re-render?

- state changes
- props changes
- context updates

Does React reload page on re-render?

✗ No.

What is Virtual DOM?

Copy of real DOM used for efficient updates.

What is diffing?

Comparing old and new Virtual DOM.

Final Revision Notes

Re-render:

- component runs again
- updates UI efficiently

Triggered by:

- state changes
- props changes
- context changes

React uses:

- Virtual DOM
- Diffing

You said:

React.memo useMemo useCallback

ChatGPT said:

React.memo, useMemo, useCallback

These are React optimization tools 🚀

Used to:

Prevent unnecessary re-renders
and improve performance.

Why Optimization Needed?

Sometimes React re-renders components unnecessarily.

This can make apps:

- slower
- less efficient

So React provides:

- React.memo
- useMemo
- useCallback

1. React.memo

What is React.memo?

React.memo prevents component re-render if props do not change.

Simple Definition

Memoized component skips unnecessary rendering.

Example Without React.memo

```
JavaScriptfunction Child() {  
  
  console.log("Child Rendered");  
  
  return <h1>Child</h1>;  
}
```

If parent re-renders:

Child also re-renders

even if props same.

Using React.memo

```
JavaScriptimport React from "react";

const Child = React.memo(function Child() {

  console.log("Child Rendered");

  return <h1>Child</h1>;
});

export default Child;
```

What Happens Now?

Child renders only when props change.

How React.memo Works

React compares:

Previous props
vs
New props

If same:

Skip re-render

Best Use Case

Use when:

- component re-renders frequently

- props rarely change

2. useMemo

What is useMemo?

useMemo memoizes calculated values.

Simple Definition

Caches expensive calculations.

Why Needed?

Heavy calculations can slow app.

useMemo stores previous result.

Syntax

```
JavaScriptconst value = useMemo(() => {  
  return calculation;  
}, [dependency]);
```

Example

```
JavaScriptimport {
  useState,
  useMemo
} from "react";

function App() {

  const [count, setCount] =
    useState(0);

  const [dark, setDark] =
    useState(false);

  const expensiveCalculation =
    useMemo(() => {

      console.log("Calculating...");

      return count * 1000;

    }, [count]);

  return (
    <div>

      <h1>{expensiveCalculation}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

      <button
        onClick={() =>
          setDark(!dark)
        }
      >
        Theme
      </button>

    </div>
  );
}
```

Important Point

Changing:

dark state

does NOT rerun calculation.

Because:

useMemo caches result

Best Use Case

Use for:

- filtering
- sorting
- large calculations
- expensive operations

3. useCallback

What is useCallback?

useCallback memoizes functions.

Simple Definition

Prevents function recreation on every render.

Problem Without useCallback

Every render creates:

new function reference

This may trigger child re-renders.

Syntax

```
JavaScriptconst func = useCallback(() => {  
  
}, [dependency]);
```

Example

```
JavaScriptimport {  
  useState,  
  useCallback  
} from "react";  
  
function App() {  
  
  const [count, setCount] =  
    useState(0);  
  
  const handleClick =  
    useCallback(() => {  
  
    console.log("Clicked");  
  
  }, []);  
  
  return (  
    <div>  
  
    <button onClick={handleClick}>  
      Click  
    </button>  
  
    <button  
      onClick={() =>  
        setCount(count + 1)  
      }  
    >  
      {count}  
    </button>  
  
    </div>  
  );  
}
```


Why useCallback Important?

Because:

Same function reused between renders.

useMemo vs useCallback

Hook

Stores

useMemo Value

useCallback Function

React.memo + useCallback Together

Very Common Interview Topic.

Example

```
JavaScriptconst Child = React.memo(({ onClick }) => {

  console.log("Child Rendered");

  return (
    <button onClick={onClick}>
      Child
    </button>
  );
});
```

Parent:

```
JavaScriptconst handleClick =
  useCallback(() => {

    console.log("Clicked");

  }, []);
```

Why Together?

Without **useCallback**:

new function created every render

So `React.memo` fails.

Important Difference

Feature	React.memo	useMemo	useCallback
Type	HOC	Hook	Hook
Prevents	Component re-render	Recalculation	Function recreation
Stores	Component result	Value	Function

Real Project Uses

React.memo

Used in:

- product cards
- large lists
- dashboard widgets

useMemo

Used in:

- filtering data
- sorting
- searching
- calculations

useCallback

Used in:

- event handlers
- optimized child components

- callback props

Common Mistakes

1. Overusing Optimization

✗ Wrong

Using everywhere unnecessarily.

Optimization also has cost.

2. Wrong Dependencies

✗ Wrong

```
JavaScriptuseMemo(() => {}, []);
```

when dependency needed.

3. Confusing useMemo & useCallback

Remember:

```
useMemo → value  
useCallback → function
```

Interview Questions

What is React.memo?

Prevents component re-render if props unchanged.

What does useMemo do?

Caches calculated values.

What does useCallback do?

Caches functions.

Difference between useMemo and useCallback?

useMemo

useCallback

Returns memoized value Returns memoized function

Why use React.memo with useCallback?

To prevent unnecessary child re-renders.

Final Revision Notes

React.memo:

- memoizes component

useMemo:

- memoizes value

useCallback:

- memoizes function

Purpose:

- performance optimization
- reduce unnecessary re-renders

You said:

state changes props changes context updates

ChatGPT said:

React Re-render Causes

React component re-renders mainly because of:

1. State Changes
2. Props Changes
3. Context Updates

1. State Changes

What is State Change?

When component state updates using:

```
setState function
```

React re-renders component.

Example

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] =
    useState(0);

  console.log("Rendered");

  return (
    <div>

      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increase
      </button>

    </div>
  );
}
```

Flow

```
Button Click
↓
setCount()
↓
State changes
↓
Component re-renders
↓
UI updates
```

Important Point

React tracks:

State changes

not normal variable changes.

Example

✗ This does NOT re-render:

```
JavaScriptlet count = 0;

count++;
```

Correct



```
JavaScriptsetCount(count + 1);
```

2. Props Changes

What are Props?

Props are data passed from:

Parent → Child

If Props Change

Child component re-renders.

Example

```
JavaScriptfunction Child({ value }) {  
  
  console.log("Child Rendered");  
  
  return <h1>{value}</h1>;  
}
```

Parent:

```
JavaScriptimport { useState } from "react";  
  
function Parent() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return (  
    <div>  
  
      <Child value={count} />  
  
      <button  
        onClick={() =>  
          setCount(count + 1)  
        }  
      >  
        Increase  
      </button>  
  
    </div>  
  );  
}
```

Flow

Parent state changes
↓
Props change
↓
Child re-renders

Important Point

Even if child UI same:

New props may trigger re-render

Preventing Unnecessary Re-render

Using:

React.memo

Example

```
JavaScriptexport default React.memo(Child);
```

3. Context Updates

What is Context?

Context used for:

Global data sharing

Example Data

- theme
- user
- language

- auth

If Context Value Changes

All consuming components re-render.

Example

```
JavaScriptimport {
  createContext,
  useContext,
  useState
} from "react";

const ThemeContext =
  createContext();

function Child() {

  const theme =
    useContext(ThemeContext);

  console.log("Child Rendered");

  return <h1>{theme}</h1>;
}
```

Parent:

```
JavaScriptfunction App() {  
  
  const [theme, setTheme] =  
    useState("Light");  
  
  return (  
  
    <ThemeContext.Provider  
      value={theme}  
    >  
  
      <Child />  
  
      <button  
        onClick={() =>  
          setTheme("Dark")  
        }  
      >  
        Change Theme  
      </button>  
  
    </ThemeContext.Provider>  
  );  
}
```

Flow

Context value changes
↓
Consumer components re-render

Comparison Table

Cause	Re-render?
State change	✓
Props change	✓
Context update	✓

Cause**Re-render?**

Normal variable change ❌

Full React Re-render Flow

User Action
↓
State / Props / Context Change
↓
React Re-renders Component
↓
Virtual DOM Updates
↓
Diffing
↓
Real DOM Updates Efficiently

Important Concept 🚀

React updates:

Only changed UI parts

not full page reload.

Optimization Hooks

Used to reduce unnecessary re-renders:

- React.memo
- useMemo
- useCallback

Real Project Examples

State Changes

- Counter
- Form input

- Todo app

Props Changes

- Parent passing product data
- Dashboard cards

Context Updates

- Dark mode
- Authentication
- Language switch

Interview Questions

What causes re-render in React?

- State changes
- Props changes
- Context updates

Does normal variable change trigger re-render?

✗ No.

What causes child re-render?

Props changes or parent re-render.

What hook handles global shared data?

```
useContext
```

Final Revision Notes

Re-render happens because of:

1. State changes
2. Props changes
3. Context updates

React does:

- Virtual DOM update
- Diffing
- Efficient DOM update

You said:

callback in setState function

ChatGPT said:

Callback in setState Function (React)

What is Callback in setState?

When updating state based on previous state, React provides:

Callback function

inside setter function.

Simple Definition

Callback receives previous state value.

Syntax

```
JavaScriptsetState((prevState) => {  
  
  return newState;  
});
```

Example with useState

```
JavaScriptimport { useState } from "react";

function App() {

  const [count, setCount] =
    useState(0);

  function increase() {

    setCount((prevCount) => {

      return prevCount + 1;

    });

  }

  return (
    <div>

      <h1>{count}</h1>

      <button onClick={increase}>
        Increase
      </button>

    </div>
  );
}
```

Output

0 → 1 → 2 → 3

What is prevCount?

Previous latest state value

Why Callback Needed?

Because React state updates are:

Asynchronous

Problem Without Callback

```
JavaScriptsetCount(count + 1);  
setCount(count + 1);
```

Expected?

+2

Actual?

Sometimes only:

+1

because both use same old value.

Correct Way

```
JavaScriptsetCount((prev) => prev + 1);  
setCount((prev) => prev + 1);
```

Output

+2 correctly

How It Works

First Update

```
prev = 0  
return 1
```

Second Update

```
prev = 1  
return 2
```

Final Result

```
count = 2
```

Flow Diagram

```
setState(callback)  
  ↓  
React gives previous state  
  ↓  
Callback returns new state  
  ↓  
React updates UI
```

Short Syntax

```
JavaScriptsetCount((prev) => prev + 1);
```

Callback with Objects

```
JavaScriptsetUser((prevUser) => {  
  
  return {  
    ...prevUser,  
    age: prevUser.age + 1  
  };  
});
```


Callback with Arrays

```
JavaScriptsetFruits((prevFruits) => {  
  
  return [...prevFruits, "Mango"];  
  
});
```

When to Use Callback Form?

Use callback when:

- ✅ new state depends on old state

Examples:

- counter
- toggle
- likes
- cart quantity

Toggle Example

```
JavaScriptsetShow((prev) => !prev);
```

Why Best Practice?

Because callback always gets:

Latest updated state

Real Project Uses

Callback state updates used in:

- counters
- shopping carts
- likes

- notifications
- toggles
- forms

Common Mistakes

1. Using Old State Directly

✗ Wrong

```
JavaScriptsetCount(count + 1);  
setCount(count + 1);
```

2. Mutating Previous State

✗ Wrong

```
JavaScriptprevUser.age++;
```

Correct



```
JavaScriptreturn {  
  ...prevUser,  
  age: prevUser.age + 1  
};
```

Difference Between Normal and Callback Update

Normal	Callback
Uses current variable	Uses latest previous state
May fail in multiple updates	Safe

Normal**Callback**

Simple updates

Best for dependent updates

Interview Questions

Why use callback in setState?

To safely update state using previous value.

What does prevState mean?

Latest previous state value.

Is state update synchronous?

✗ No, asynchronous.

When should callback form be used?

When new state depends on old state.

Final Revision Notes

Callback in setState:

- receives previous state
- used for safe updates

Syntax:

```
setState((prev) => newValue)
```

Best for:

- counters
- toggles
- dependent updates

You said:

ChatGPT said:

Advanced Concepts of State in React

1. What is State?

State stores dynamic data in a component.

State changes over time.

Examples:

- counter
- form input
- dark mode
- cart items

2. Local State

State inside one component only.

Example

```
JavaScriptfunction App() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return <h1>{count}</h1>;  
}
```

Important Point

Only this component can use this state.

3. Shared State

When multiple components need same state.

Solution:

Lift state up

Example

Parent stores state and passes via props.

```
JavaScriptfunction Parent() {  
  
  const [count, setCount] =  
    useState(0);  
  
  return (  
    <>  
      <Child1 count={count} />  
      <Child2 count={count} />  
    </>  
  );  
}
```

4. Derived State

State calculated from another state.

Example

```
JavaScriptconst [price] = useState(100);  
const tax = price * 0.18;
```

Important

Avoid storing unnecessary duplicate state.

Wrong ❌

```
JavaScriptconst [fullName, setFullName] =  
  useState(first + last);
```

Better

```
JavaScriptconst fullName = first + last;
```

5. Immutable State

React state should NEVER mutate directly.

Wrong

```
JavaScriptuser.name = "Ali";
```

Correct

```
JavaScriptsetUser({  
  ...user,  
  name: "Ali"  
});
```

Why?

Because React checks:

```
Reference changes
```

to detect updates.

6. State Batching

React combines multiple state updates.

Example

```
JavaScriptsetCount((prev) => prev + 1);  
setCount((prev) => prev + 1);
```

Result

Only one re-render

This improves performance.

7. Asynchronous State Updates

State updates are NOT immediate.

Example

```
JavaScriptsetCount(count + 1);  
  
console.log(count);
```

Output

May show old value.

Why?

Because React schedules updates.

8. Functional Updates

Best Practice 🚀

Used when new state depends on old state.

Example

```
JavaScriptsetCount((prev) => prev + 1);
```

Why Important?

Always uses latest state value.

9. Lazy Initial State

Used for expensive initial calculations.

Normal

```
JavaScriptconst [value, setValue] =  
  useState(expensiveFunction());
```

Runs every render.

Better

```
JavaScriptconst [value, setValue] =  
  useState(() => expensiveFunction());
```

Runs only once.

10. State and Re-render

Whenever state changes:

Component re-renders

Flow

```
State Update  
↓  
Component Function Runs Again  
↓  
Virtual DOM Updates  
↓  
Real DOM Updates
```

11. State with Arrays

Add Item

```
JavaScriptsetItems([...items, newItem]);
```


Remove Item

```
JavaScriptsetItems(  
  items.filter(  
    (item) => item.id !== id  
  )  
);
```

Update Item

```
JavaScriptsetItems(  
  items.map((item) =>  
    item.id === id  
      ? { ...item, done: true }  
      : item  
  )  
);
```

12. State with Objects

Example

```
JavaScriptconst [user, setUser] =  
  useState({  
    name: "Aamir",  
    age: 21  
  });
```

Update Object

```
JavaScriptsetUser({  
  ...user,  
  age: 22  
});
```

13. Controlled Components

Input controlled using state.

Example

```
JavaScriptconst [name, setName] =  
  useState("");  
  
<input  
  value={name}  
  onChange={(e) =>  
    setName(e.target.value)  
  }  
</>
```

Benefits

✅ Single source of truth ✅ Easier validation ✅ Better form handling

14. Lifting State Up

Move shared state to parent component.

Example

```
JavaScriptParent  
├── Child A  
└── Child B
```

Parent controls shared state.

15. Global State

When many components need same data.

Used with:

- Context API
- Redux
- Zustand

Example Data

- user login
- theme

- language
- cart

16. useReducer vs useState

useState	useReducer
Simple state	Complex state
Easy	More structured
Small apps	Large apps

17. State Persistence

Normally state resets on refresh.

To persist:

- localStorage
- sessionStorage
- database

Example

```
JavaScriptlocalStorage.setItem("theme", "dark");
```

18. State Reset

State resets when component unmounts.

Example

```
Component removed  
↓  
State destroyed
```

19. State and Keys

Changing component key resets state.

Example

```
JavaScript<Component key={id} />
```

New key:

```
New component instance
```

20. Avoid Too Much State

Not everything should be state.

Wrong ❌

```
JavaScriptconst [fullName, setFullName]
```

if derived from existing values.

Better

```
JavaScriptconst fullName =  
  firstName + lastName;
```

Common State Mistakes 🚀

Direct Mutation ❌

```
JavaScriptarr.push(item);
```

Correct

```
JavaScriptsetArr([...arr, item]);
```

Infinite Re-render ❌

```
JavaScriptsetCount(count + 1);
```

inside component body.

Missing Functional Update ❌

```
JavaScriptsetCount(count + 1);  
setCount(count + 1);
```

Correct

```
JavaScriptsetCount((prev) => prev + 1);
```

Real Project Uses 🚀

State used in:

- authentication
- shopping cart
- forms
- notifications
- dark mode
- API data
- chat apps

Most Asked Interview Questions

Why React state immutable?

Because React detects updates using reference changes.

Why state updates asynchronous?

For performance optimization and batching.

Difference between local and global state?

Local

Global

One component Multiple components

What is lifting state up?

Moving shared state to parent.

Why functional updates important?

To safely use latest previous state.

Final Revision Notes

Advanced State Concepts:

- immutable state
- batching
- async updates
- functional updates
- derived state
- lifting state up
- global state
- controlled components

ChatGPT can make mistakes. Check important info.